

ECEN 662: Estimation & Detection Theory

Lecture 13: Variational Inference

Tie Liu

Texas A&M University

The reality check: Intractable posteriors

- In previous lectures, we relied on Bayes' rule to compute the exact posterior:

$$\pi(\theta|x) = \frac{\pi(\theta)f_{\theta}(x)}{\int_{\Omega} \pi(\theta)f_{\theta}(x)d\theta}$$

The reality check: Intractable posteriors

- In previous lectures, we relied on Bayes' rule to compute the exact posterior:

$$\pi(\theta|x) = \frac{\pi(\theta)f_{\theta}(x)}{\int_{\Omega} \pi(\theta)f_{\theta}(x)d\theta}$$

- For complex real-world models (e.g., hierarchical models or Bayesian neural networks), the denominator—the *evidence* $m(x)$ —requires integrating over thousands of dimensions. This is computationally *intractable*.

The reality check: Intractable posteriors

- In previous lectures, we relied on Bayes' rule to compute the exact posterior:

$$\pi(\theta|x) = \frac{\pi(\theta)f_{\theta}(x)}{\int_{\Omega} \pi(\theta)f_{\theta}(x)d\theta}$$

- For complex real-world models (e.g., hierarchical models or Bayesian neural networks), the denominator—the *evidence* $m(x)$ —requires integrating over thousands of dimensions. This is computationally *intractable*.
- When exact inference fails, we generally have two paths forward:
 - **Stochastic Sampling (MCMC):** Draw samples directly from the true posterior (accurate, but often exceptionally slow).
 - **Variational Inference (VI):** Turn the integration problem into an *optimization* problem (approximate, but fast and scalable).

The reality check: Intractable posteriors

- In previous lectures, we relied on Bayes' rule to compute the exact posterior:

$$\pi(\theta|x) = \frac{\pi(\theta)f_{\theta}(x)}{\int_{\Omega} \pi(\theta)f_{\theta}(x)d\theta}$$

- For complex real-world models (e.g., hierarchical models or Bayesian neural networks), the denominator—the *evidence* $m(x)$ —requires integrating over thousands of dimensions. This is computationally *intractable*.
- When exact inference fails, we generally have two paths forward:
 - **Stochastic Sampling (MCMC):** Draw samples directly from the true posterior (accurate, but often exceptionally slow).
 - **Variational Inference (VI):** Turn the integration problem into an *optimization* problem (approximate, but fast and scalable).
- **The Core Idea of VI:** Instead of computing the highly complex true posterior $\pi(\theta|x)$, we define a family of simpler, well-understood distributions $q_{\phi}(\theta)$. We then optimize the parameters ϕ to make $q_{\phi}(\theta)$ as *close* to $\pi(\theta|x)$ as possible.

Example: The Gaussian Mixture Model (GMM)

- We wish to group data into K clusters. We face two sets of *unknown* parameters:
 - **The Centers:** $\mu = (\mu_1, \dots, \mu_K)$.
 - **The Assignments:** n unlabeled data points $X = (X_1, \dots, X_n)$, each originating from an unknown cluster C_i .

Example: The Gaussian Mixture Model (GMM)

- We wish to group data into K clusters. We face two sets of *unknown* parameters:
 - **The Centers:** $\mu = (\mu_1, \dots, \mu_K)$.
 - **The Assignments:** n unlabeled data points $X = (X_1, \dots, X_n)$, each originating from an unknown cluster C_i .
- **The Generative Model:** Given the cluster assignment C_i and the location of its center μ_{C_i} , each point is drawn independently:

$$X_i \sim \mathcal{N}(\mu_{C_i}, 1)$$

Example: The Gaussian Mixture Model (GMM)

- We wish to group data into K clusters. We face two sets of *unknown* parameters:
 - **The Centers:** $\mu = (\mu_1, \dots, \mu_K)$.
 - **The Assignments:** n unlabeled data points $X = (X_1, \dots, X_n)$, each originating from an unknown cluster C_i .
- **The Generative Model:** Given the cluster assignment C_i and the location of its center μ_{C_i} , each point is drawn independently:

$$X_i \sim \mathcal{N}(\mu_{C_i}, 1)$$

- **The Bayesian Priors:** We treat both the assignments C and the centers μ as independent random variables:

Example: The Gaussian Mixture Model (GMM)

- We wish to group data into K clusters. We face two sets of *unknown* parameters:
 - **The Centers:** $\mu = (\mu_1, \dots, \mu_K)$.
 - **The Assignments:** n unlabeled data points $X = (X_1, \dots, X_n)$, each originating from an unknown cluster C_i .
- **The Generative Model:** Given the cluster assignment C_i and the location of its center μ_{C_i} , each point is drawn independently:

$$X_i \sim \mathcal{N}(\mu_{C_i}, 1)$$

- **The Bayesian Priors:** We treat both the assignments C and the centers μ as independent random variables:
 - *Assignment Prior:* Each point is equally likely to be in any cluster.

$$C_i \sim \mathcal{U}([K])$$

Example: The Gaussian Mixture Model (GMM)

- We wish to group data into K clusters. We face two sets of *unknown* parameters:
 - **The Centers:** $\mu = (\mu_1, \dots, \mu_K)$.
 - **The Assignments:** n unlabeled data points $X = (X_1, \dots, X_n)$, each originating from an unknown cluster C_i .
- **The Generative Model:** Given the cluster assignment C_i and the location of its center μ_{C_i} , each point is drawn independently:

$$X_i \sim \mathcal{N}(\mu_{C_i}, 1)$$

- **The Bayesian Priors:** We treat both the assignments C and the centers μ as independent random variables:
 - *Assignment Prior:* Each point is equally likely to be in any cluster.

$$C_i \sim \mathcal{U}([K])$$

- *Center Prior:* The centers are drawn independently from a Gaussian.

$$\mu_k \sim \mathcal{N}(0, \sigma^2)$$

The Computational Trap: The Exact Posterior

- To solve the GMM, we target the joint posterior of the assignments c and centers μ :

$$f(c, \mu | x) = \frac{p(c)f(\mu) \prod_{i=1}^n f(x_i | \mu_{c_i})}{m(x)}$$

The Computational Trap: The Exact Posterior

- To solve the GMM, we target the joint posterior of the assignments c and centers μ :

$$f(c, \mu | x) = \frac{p(c) f(\mu) \prod_{i=1}^n f(x_i | \mu_{c_i})}{m(x)}$$

- The *evidence* $m(x)$ requires marginalizing out all unknowns (summing over discrete c and integrating over continuous μ):

$$m(x) = \sum_c p(c) \int f(\mu) \prod_{i=1}^n f(x_i | \mu_{c_i}) d\mu$$

The Computational Trap: The Exact Posterior

- To solve the GMM, we target the joint posterior of the assignments c and centers μ :

$$f(c, \mu | x) = \frac{p(c) f(\mu) \prod_{i=1}^n f(x_i | \mu_{c_i})}{m(x)}$$

- The *evidence* $m(x)$ requires marginalizing out all unknowns (summing over discrete c and integrating over continuous μ):

$$m(x) = \sum_c p(c) \int f(\mu) \prod_{i=1}^n f(x_i | \mu_{c_i}) d\mu$$

- **The Catch:** Thanks to the *conjugacy* between the Gaussian prior and Gaussian likelihood, the integral itself is easily computable.
- However, because each of the n points can belong to K clusters, there are K^n terms in the outer sum.

The Computational Trap: The Exact Posterior

- To solve the GMM, we target the joint posterior of the assignments c and centers μ :

$$f(c, \mu | x) = \frac{p(c) f(\mu) \prod_{i=1}^n f(x_i | \mu_{c_i})}{m(x)}$$

- The *evidence* $m(x)$ requires marginalizing out all unknowns (summing over discrete c and integrating over continuous μ):

$$m(x) = \sum_c p(c) \int f(\mu) \prod_{i=1}^n f(x_i | \mu_{c_i}) d\mu$$

- **The Catch:** Thanks to the *conjugacy* between the Gaussian prior and Gaussian likelihood, the integral itself is easily computable.
- However, because each of the n points can belong to K clusters, there are K^n terms in the outer sum.
- **Conclusion:** Computing the evidence scales *exponentially* in n . It is strictly intractable for real data, forcing us to use *approximate* methods like Variational Inference.

The Classical Solution: MCMC

- For decades, the dominant paradigm for *approximate* inference has been *Markov chain Monte Carlo (MCMC)*.

The Classical Solution: MCMC

- For decades, the dominant paradigm for *approximate* inference has been *Markov chain Monte Carlo (MCMC)*.
- **The MCMC Pipeline:**
 - **Construct:** Build an ergodic Markov chain on Θ whose stationary distribution is exactly the true posterior $f(\theta|x)$.
 - **Sample:** Simulate the chain for a long time to collect samples.
 - **Approximate:** Use an empirical histogram of the collected samples to represent the posterior.

The Classical Solution: MCMC

- For decades, the dominant paradigm for *approximate* inference has been *Markov chain Monte Carlo (MCMC)*.
- **The MCMC Pipeline:**
 - **Construct:** Build an ergodic Markov chain on Θ whose stationary distribution is exactly the true posterior $f(\theta|x)$.
 - **Sample:** Simulate the chain for a long time to collect samples.
 - **Approximate:** Use an empirical histogram of the collected samples to represent the posterior.
- Algorithms like *Metropolis-Hastings* and the *Gibbs Sampler* are indispensable. Given infinite time, MCMC is asymptotically exact.

The Classical Solution: MCMC

- For decades, the dominant paradigm for *approximate* inference has been *Markov chain Monte Carlo (MCMC)*.
- **The MCMC Pipeline:**
 - **Construct:** Build an ergodic Markov chain on Θ whose stationary distribution is exactly the true posterior $f(\theta|x)$.
 - **Sample:** Simulate the chain for a long time to collect samples.
 - **Approximate:** Use an empirical histogram of the collected samples to represent the posterior.
- Algorithms like *Metropolis-Hastings* and the *Gibbs Sampler* are indispensable. Given infinite time, MCMC is asymptotically exact.
- **The Catch:** MCMC is inherently sequential and often *highly computationally expensive*. For massive modern datasets or complex models, MCMC is simply too slow. We need a faster approach.

Variational Inference (VI)

Rather than use sampling, the main idea behind *variational inference (VI)* is to use *optimization*:

- **Propose a Variational Family:** Posit a family $\mathcal{Q}$ of approximate densities $q(\theta)$ over the unknown parameters.

Variational Inference (VI)

Rather than use sampling, the main idea behind *variational inference (VI)* is to use *optimization*:

- **Propose a Variational Family:** Posit a family $\mathcal{Q}$ of approximate densities $q(\theta)$ over the unknown parameters.
- **Minimize Divergence:** Find the member of $\mathcal{Q}$ that is "closest" to the exact posterior using the *Kullback-Leibler (KL) divergence*:

$$q^* = \arg \min_{q \in \mathcal{Q}} D_{KL}(q(\theta) \parallel f(\theta|x))$$

Variational Inference (VI)

Rather than use sampling, the main idea behind *variational inference (VI)* is to use *optimization*:

- **Propose a Variational Family:** Posit a family $\mathcal{Q}$ of approximate densities $q(\theta)$ over the unknown parameters.
- **Minimize Divergence:** Find the member of $\mathcal{Q}$ that is "closest" to the exact posterior using the *Kullback-Leibler (KL) divergence*:

$$q^* = \arg \min_{q \in \mathcal{Q}} D_{KL}(q(\theta) \parallel f(\theta|x))$$

- **Approximate:** Use the optimized $q^*(\theta)$ as the surrogate for the true posterior $f(\theta|x)$.
- *Note:* This allows us to use standard optimization tools like **Stochastic Gradient Descent**, making inference incredibly fast.

Comparison: VI vs. MCMC

Choosing between these paradigms is a trade-off between *precision* and *speed*:

- **MCMC:** The "Gold Standard."
 - **Pro:** Asymptotically exact samples from the target.
 - **Con:** Computationally intensive and difficult to scale.

Comparison: VI vs. MCMC

Choosing between these paradigms is a trade-off between *precision* and *speed*:

- **MCMC:** The "Gold Standard."
 - **Pro:** Asymptotically exact samples from the target.
 - **Con:** Computationally intensive and difficult to scale.
- **VI:** The "Engineers' Choice."
 - **Pro:** Extremely fast; utilizes *stochastic and distributed optimization* (SGD).
 - **Con:** No exactness guarantees; accuracy is limited by the choice of family Q .

Comparison: VI vs. MCMC

Choosing between these paradigms is a trade-off between *precision* and *speed*:

- **MCMC:** The "Gold Standard."
 - **Pro:** Asymptotically exact samples from the target.
 - **Con:** Computationally intensive and difficult to scale.
- **VI:** The "Engineers' Choice."
 - **Pro:** Extremely fast; utilizes *stochastic and distributed optimization* (SGD).
 - **Con:** No exactness guarantees; accuracy is limited by the choice of family $\mathcal{Q}$.
- **Selection Criteria:**
 - Use **VI** for large-scale data and rapid model exploration.
 - Use **MCMC** for small data where precision is worth the high cost.

Comparison: VI vs. MCMC

Choosing between these paradigms is a trade-off between *precision* and *speed*:

- **MCMC:** The "Gold Standard."
 - **Pro:** Asymptotically exact samples from the target.
 - **Con:** Computationally intensive and difficult to scale.
- **VI:** The "Engineers' Choice."
 - **Pro:** Extremely fast; utilizes *stochastic and distributed optimization* (SGD).
 - **Con:** No exactness guarantees; accuracy is limited by the choice of family Q .
- **Selection Criteria:**
 - Use **VI** for large-scale data and rapid model exploration.
 - Use **MCMC** for small data where precision is worth the high cost.
- *Note:* The relative accuracy of VI vs. MCMC in finite-time scenarios remains an active area of research.

The Evidence Lower Bound (ELBO)

- We seek to solve the optimization problem:

$$q^* = \arg \min_{q \in \mathcal{Q}} D_{KL}(q(\theta) \parallel f(\theta|x))$$

The Evidence Lower Bound (ELBO)

- We seek to solve the optimization problem:

$$q^* = \arg \min_{q \in \mathcal{Q}} D_{KL}(q(\theta) \parallel f(\theta|x))$$

- **The Computational Wall:** We cannot compute this objective directly because it depends on the *evidence* $m(x)$.

The Evidence Lower Bound (ELBO)

- We seek to solve the optimization problem:

$$q^* = \arg \min_{q \in \mathcal{Q}} D_{KL}(q(\theta) \parallel f(\theta|x))$$

- **The Computational Wall:** We cannot compute this objective directly because it depends on the *evidence* $m(x)$.
- **The Decomposition:** Expanding the KL divergence using $f(\theta|x) = \frac{f(\theta, x)}{m(x)}$:

$$\begin{aligned} D_{KL}(q(\theta) \parallel f(\theta|x)) &= \mathbb{E}_q[\log q(\Theta)] - \mathbb{E}_q[\log f(\Theta|x)] \\ &= \underbrace{\mathbb{E}_q[\log q(\Theta)] - \mathbb{E}_q[\log f(\Theta, x)]}_{\text{Computable}} + \underbrace{\log m(x)}_{\text{Intractable Constant}} \end{aligned}$$

where all expectations are over $\Theta \sim q(\theta)$.

- **The Strategy:** Since $\log m(x)$ is constant with respect to q , we can ignore it during optimization.

The Evidence Lower Bound (ELBO)

- Since we cannot minimize the KL divergence directly, we maximize a computable surrogate objective:

$$ELBO(q) = \mathbb{E}_q[\log f(\Theta, x)] - \mathbb{E}_q[\log q(\Theta)]$$

The Evidence Lower Bound (ELBO)

- Since we cannot minimize the KL divergence directly, we maximize a computable surrogate objective:

$$ELBO(q) = \mathbb{E}_q[\log f(\Theta, x)] - \mathbb{E}_q[\log q(\Theta)]$$

- **The Optimization Link:** Recall the identity from the previous slide:

$$\log m(x) = ELBO(q) + D_{KL}(q(\theta) \parallel f(\theta|x))$$

Since $\log m(x)$ is constant with respect to q , *maximizing the ELBO* is exactly equivalent to *minimizing the KL divergence*.

The Evidence Lower Bound (ELBO)

- Since we cannot minimize the KL divergence directly, we maximize a computable surrogate objective:

$$ELBO(q) = \mathbb{E}_q[\log f(\Theta, x)] - \mathbb{E}_q[\log q(\Theta)]$$

- **The Optimization Link:** Recall the identity from the previous slide:

$$\log m(x) = ELBO(q) + D_{KL}(q(\theta) \parallel f(\theta|x))$$

Since $\log m(x)$ is constant with respect to q , *maximizing the ELBO* is exactly equivalent to *minimizing the KL divergence*.

- **The "Lower Bound":** Because $D_{KL} \geq 0$, it follows that:

$$ELBO(q) \leq \log m(x)$$

for any q . The ELBO provides a *rigorous lower bound* on the log-evidence. By "tightening" this bound, we improve our posterior approximation.

The Mean-field Variational Family

- To complete our optimization problem, we must specify the family $\mathcal{Q}$.

The Mean-field Variational Family

- To complete our optimization problem, we must specify the family $\mathcal{Q}$.
- **The Trade-off:**
 - *More Complex $\mathcal{Q}$:* Better approximation, but much harder to optimize.
 - *Simpler $\mathcal{Q}$:* Easier optimization, but potentially less accurate.

The Mean-field Variational Family

- To complete our optimization problem, we must specify the family $\mathcal{Q}$.
- **The Trade-off:**
 - *More Complex $\mathcal{Q}$:* Better approximation, but much harder to optimize.
 - *Simpler $\mathcal{Q}$:* Easier optimization, but potentially less accurate.
- **The Mean-field Assumption:** We assume that all latent variables are mutually *independent*:

$$q(\theta) = \prod_{j=1}^m q_j(\theta_j)$$

- Each unknown θ_j is governed by its own factor q_j . During optimization, we update these factors iteratively to maximize the ELBO.
- *Key Insight:* This turns a high-dimensional global optimization into a sequence of simpler, low-dimensional coordinate updates.

Beyond Mean-Field: Increasing Fidelity

- While the mean-field family is our workhorse, we can expand $\mathcal{Q}$ to capture more complex posterior structures:
 - **Structured VI:** Allows for *dependencies* between certain latent variables (e.g., preserving a chain structure).
 - **Mixture Models:** Uses a *mixture of densities* to approximate multi-modal posteriors.

Beyond Mean-Field: Increasing Fidelity

- While the mean-field family is our workhorse, we can expand Q to capture more complex posterior structures:
 - **Structured VI:** Allows for *dependencies* between certain latent variables (e.g., preserving a chain structure).
 - **Mixture Models:** Uses a *mixture of densities* to approximate multi-modal posteriors.
- **The Trade-off:**
 - **Pro:** Improved fidelity; the approximation can better "mimic" the true posterior.
 - **Con:** Leads to a much more *difficult optimization problem*. You lose the simplicity of independent coordinate updates.

Coordinate Ascent Variational Inference (CAVI)

- We now have a formal optimization problem. To solve it, we use *Coordinate Ascent Variational Inference (CAVI)*.

Coordinate Ascent Variational Inference (CAVI)

- We now have a formal optimization problem. To solve it, we use *Coordinate Ascent Variational Inference (CAVI)*.
- **The Iterative Approach:**
 - Optimize one factor q_j at a time while holding all others *fixed*.
 - Successively "climb" the ELBO until reaching a *local optimum*.

Coordinate Ascent Variational Inference (CAVI)

- We now have a formal optimization problem. To solve it, we use *Coordinate Ascent Variational Inference (CAVI)*.
- **The Iterative Approach:**
 - Optimize one factor q_j at a time while holding all others *fixed*.
 - Successively "climb" the ELBO until reaching a *local optimum*.
- **The Coordinate Update:** For the j -th parameter θ_j , we isolate its contribution to the ELBO:

$$ELBO(q_j) = \mathbb{E}_j[\mathbb{E}_{-j}[\log f(\Theta_j, \Theta_{-j}, x)]] - \mathbb{E}_j[\log q_j(\Theta_j)] + \text{const}$$

where $\mathbb{E}_{-j}$ denotes the expectation over all factors $\ell \neq j$.

Coordinate Ascent Variational Inference (CAVI)

- We now have a formal optimization problem. To solve it, we use *Coordinate Ascent Variational Inference (CAVI)*.
- **The Iterative Approach:**
 - Optimize one factor q_j at a time while holding all others *fixed*.
 - Successively "climb" the ELBO until reaching a *local optimum*.
- **The Coordinate Update:** For the j -th parameter θ_j , we isolate its contribution to the ELBO:

$$ELBO(q_j) = \mathbb{E}_j [\mathbb{E}_{-j} [\log f(\Theta_j, \Theta_{-j}, x)]] - \mathbb{E}_j [\log q_j(\Theta_j)] + \text{const}$$

where $\mathbb{E}_{-j}$ denotes the expectation over all factors $\ell \neq j$.

- **The Goal:** Find the q_j that maximizes this expression. As we will see, for many models, this leads to a very clean, closed-form update rule.

The Optimal Coordinate Update

- What is the optimal shape for the factor q_j ?
- **The Target Density:** Let $q_j^*(\theta_j)$ be defined as:

$$q_j^*(\theta_j) \propto \exp \{ \mathbb{E}_{-j} [\log f(\theta_j, \Theta_{-j}, x)] \}$$

The Optimal Coordinate Update

- What is the optimal shape for the factor q_j ?
- **The Target Density:** Let $q_j^*(\theta_j)$ be defined as:

$$q_j^*(\theta_j) \propto \exp \{ \mathbb{E}_{-j} [\log f(\theta_j, \Theta_{-j}, x)] \}$$

- **Maximizing the ELBO:** Rearranging the objective reveals:

$$ELBO(q_j) = -D_{KL}(q_j(\theta_j) \parallel q_j^*(\theta_j)) + \text{Const} \leq \text{Const}$$

where equality holds (the maximum is reached) *if and only if* $q_j = q_j^*$.

The Optimal Coordinate Update

- What is the optimal shape for the factor q_j ?
- **The Target Density:** Let $q_j^*(\theta_j)$ be defined as:

$$q_j^*(\theta_j) \propto \exp \{ \mathbb{E}_{-j} [\log f(\theta_j, \Theta_{-j}, x)] \}$$

- **Maximizing the ELBO:** Rearranging the objective reveals:

$$ELBO(q_j) = -D_{KL}(q_j(\theta_j) \parallel q_j^*(\theta_j)) + \text{Const} \leq \text{Const}$$

where equality holds (the maximum is reached) *if and only if* $q_j = q_j^*$.

- **The CAVI Update Rule:** To update the factor q_j , we set:

$$q_j^*(\theta_j) \propto \exp \{ \mathbb{E}_{-j} [\log f(\theta_j, \Theta_{-j}, x)] \}$$

- *Intuition:* The optimal factor q_j is the *exponentiated average* of the log-joint distribution over all other variables.

CAVI for Gaussian Mixture Models

- We now apply the general update rule to our clustering example:
- **Update Assignments (C_i):** For each point i , the probability $\psi_{i,k}$ of belonging to cluster k is:

$$\psi_{i,k} \propto \exp\left(-\frac{\mathbb{E}_q[(x_i - \mu_k)^2]}{2}\right)$$

CAVI for Gaussian Mixture Models

- We now apply the general update rule to our clustering example:
- **Update Assignments (C_i):** For each point i , the probability $\psi_{i,k}$ of belonging to cluster k is:

$$\psi_{i,k} \propto \exp \left(-\frac{\mathbb{E}_q[(x_i - \mu_k)^2]}{2} \right)$$

- **Update Centers (μ_k):** The new variational factor $q(\mu_k)$ is a Gaussian:

$$q(\mu_k) = \mathcal{N} \left(\frac{\sum_i \psi_{i,k} x_i}{1/\sigma^2 + \sum_i \psi_{i,k}}, \frac{1}{1/\sigma^2 + \sum_i \psi_{i,k}} \right)$$

CAVI for Gaussian Mixture Models

- We now apply the general update rule to our clustering example:
- **Update Assignments (C_i):** For each point i , the probability $\psi_{i,k}$ of belonging to cluster k is:

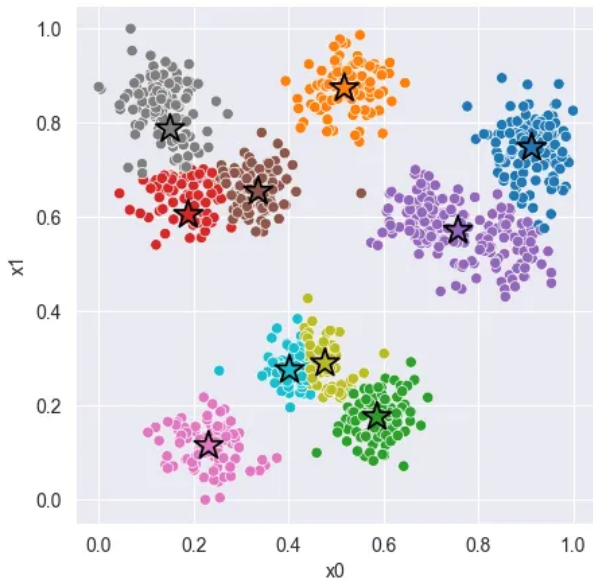
$$\psi_{i,k} \propto \exp \left(-\frac{\mathbb{E}_q[(x_i - \mu_k)^2]}{2} \right)$$

- **Update Centers (μ_k):** The new variational factor $q(\mu_k)$ is a Gaussian:

$$q(\mu_k) = \mathcal{N} \left(\frac{\sum_i \psi_{i,k} x_i}{1/\sigma^2 + \sum_i \psi_{i,k}}, \frac{1}{1/\sigma^2 + \sum_i \psi_{i,k}} \right)$$

- **The Connection:** This iterative process—assigning points to centers and then re-calculating centers based on those assignments—is a *probabilistic* generalization of the *K-means* algorithm.
- *Key Benefit:* Unlike K-means, VI provides a full measure of *uncertainty* for both the assignments and the cluster locations.

K-means clustering



Black Box Variational Inference

- While CAVI offers closed-form solutions for specific models, it is not generalizable.

Black Box Variational Inference

- While CAVI offers closed-form solutions for specific models, it is not generalizable.
- **General Parameterization:** For complex models, we parameterize the variational distribution $q_{\lambda}(\theta)$ by a vector of *variational parameters* λ .

Black Box Variational Inference

- While CAVI offers closed-form solutions for specific models, it is not generalizable.
- **General Parameterization:** For complex models, we parameterize the variational distribution $q_\lambda(\theta)$ by a vector of *variational parameters* λ .
- **Optimization Problem:** We maximize the ELBO with respect to λ :

$$\max_{\lambda} (\mathbb{E}_{q_\lambda} [\log f(\Theta, x)] - \mathbb{E}_{q_\lambda} [\log q_\lambda(\Theta)])$$

- This can be solved using *Stochastic Gradient Ascent*, treating the model as a “black box” that only requires log-likelihoods and gradients.

Black Box Variational Inference

- While CAVI offers closed-form solutions for specific models, it is not generalizable.
- **General Parameterization:** For complex models, we parameterize the variational distribution $q_\lambda(\theta)$ by a vector of *variational parameters* λ .
- **Optimization Problem:** We maximize the ELBO with respect to λ :

$$\max_{\lambda} (\mathbb{E}_{q_\lambda} [\log f(\Theta, x)] - \mathbb{E}_{q_\lambda} [\log q_\lambda(\Theta)])$$

- This can be solved using *Stochastic Gradient Ascent*, treating the model as a “black box” that only requires log-likelihoods and gradients.
- *Key Challenge:* We must now compute the *gradient* of the ELBO with respect to λ , which involves differentiating an expectation.

Computing the Gradient of the ELBO

- To optimize the ELBO, we need its gradient with respect to λ :

$$ELBO(\lambda) = \int_{\Omega} q_{\lambda}(\theta) (\log f(\theta, x) - \log q_{\lambda}(\theta)) d\theta$$

Computing the Gradient of the ELBO

- To optimize the ELBO, we need its gradient with respect to λ :

$$ELBO(\lambda) = \int_{\Omega} q_{\lambda}(\theta) (\log f(\theta, x) - \log q_{\lambda}(\theta)) d\theta$$

- Applying the *"log-derivative trick"* ($\nabla q_{\lambda} = q_{\lambda} \nabla \log q_{\lambda}$) and using the fact that $\mathbb{E}_{q_{\lambda}}[\nabla_{\lambda} \log q_{\lambda}(\Theta)] = 0$:

$$\nabla_{\lambda} ELBO(\lambda) = \mathbb{E}_{q_{\lambda}} [(\nabla_{\lambda} \log q_{\lambda}(\Theta)) (\log f(\Theta, x) - \log q_{\lambda}(\Theta))]$$

Computing the Gradient of the ELBO

- To optimize the ELBO, we need its gradient with respect to λ :

$$ELBO(\lambda) = \int_{\Omega} q_{\lambda}(\theta) (\log f(\theta, x) - \log q_{\lambda}(\theta)) d\theta$$

- Applying the *"log-derivative trick"* ($\nabla q_{\lambda} = q_{\lambda} \nabla \log q_{\lambda}$) and using the fact that $\mathbb{E}_{q_{\lambda}}[\nabla_{\lambda} \log q_{\lambda}(\Theta)] = 0$:

$$\nabla_{\lambda} ELBO(\lambda) = \mathbb{E}_{q_{\lambda}} [(\nabla_{\lambda} \log q_{\lambda}(\Theta)) (\log f(\Theta, x) - \log q_{\lambda}(\Theta))]$$

- **Implementation:** This true gradient is intractable. However, we can generate a noisy, *unbiased estimate* by sampling $\theta^{(s)} \sim q_{\lambda}$ and computing:

$$\nabla_{\lambda} ELBO(\lambda) \approx \frac{1}{S} \sum_{s=1}^S \left[\left(\nabla_{\lambda} \log q_{\lambda}(\theta^{(s)}) \right) \left(\log f(\theta^{(s)}, x) - \log q_{\lambda}(\theta^{(s)}) \right) \right]$$

The Reparameterization Trick

- **The Limitation of Score Functions:** The gradient estimator in the previous slide often has *high variance*, leading to slow convergence in stochastic gradient ascent.

The Reparameterization Trick

- **The Limitation of Score Functions:** The gradient estimator in the previous slide often has *high variance*, leading to slow convergence in stochastic gradient ascent.
- **The Trick:** To reduce variance, express the random variable Θ as a deterministic, differentiable function of parameters λ and an auxiliary noise variable ϵ with a *fixed* distribution $p(\epsilon)$:

$$\Theta = g(\lambda, \epsilon) \quad \text{where} \quad \epsilon \sim p(\epsilon)$$

The Reparameterization Trick

- **The Limitation of Score Functions:** The gradient estimator in the previous slide often has *high variance*, leading to slow convergence in stochastic gradient ascent.
- **The Trick:** To reduce variance, express the random variable Θ as a deterministic, differentiable function of parameters λ and an auxiliary noise variable ϵ with a *fixed* distribution $p(\epsilon)$:

$$\Theta = g(\lambda, \epsilon) \quad \text{where} \quad \epsilon \sim p(\epsilon)$$

- **Example (Gaussian):** If $\Theta \sim \mathcal{N}(\mu, \sigma^2)$, we set $\lambda = (\mu, \sigma)$ and let:

$$\Theta = \mu + \sigma \odot \epsilon \quad \text{where} \quad \epsilon \sim \mathcal{N}(0, I)$$

- **Benefit:** We can propagate gradients directly through $g(\lambda, \epsilon)$, leading to much *lower variance* estimators. This is the foundation of *Variational Autoencoders (VAEs)*.

Black Box VI Algorithm (Reparameterized)

- **Objective:** Maximize the ELBO with respect to variational parameters λ :

$$ELBO(\lambda) = \mathbb{E}_{\epsilon \sim p(\epsilon)} [\log f(g(\lambda, \epsilon), x) - \log q_{\lambda}(g(\lambda, \epsilon))]$$

Black Box VI Algorithm (Reparameterized)

- **Objective:** Maximize the ELBO with respect to variational parameters λ :

$$ELBO(\lambda) = \mathbb{E}_{\epsilon \sim p(\epsilon)} [\log f(g(\lambda, \epsilon), x) - \log q_{\lambda}(g(\lambda, \epsilon))]$$

- **Stochastic Gradient Ascent Algorithm:**

1. **Initialize** λ .
2. **Loop** until convergence:
 - 2.1 Sample noise $\epsilon \sim p(\epsilon)$.
 - 2.2 Compute sample: $\theta = g(\lambda, \epsilon)$.
 - 2.3 Compute gradient estimate: $\hat{g} = \nabla_{\lambda} [\log f(\theta, x) - \log q_{\lambda}(\theta)]$.
 - 2.4 Update parameters: $\lambda \leftarrow \lambda + \rho \cdot \hat{g}$ (where ρ is learning rate).

Black Box VI Algorithm (Reparameterized)

- **Objective:** Maximize the ELBO with respect to variational parameters λ :

$$ELBO(\lambda) = \mathbb{E}_{\epsilon \sim p(\epsilon)} [\log f(g(\lambda, \epsilon), x) - \log q_\lambda(g(\lambda, \epsilon))]$$

- **Stochastic Gradient Ascent Algorithm:**

1. **Initialize** λ .
2. **Loop** until convergence:
 - 2.1 Sample noise $\epsilon \sim p(\epsilon)$.
 - 2.2 Compute sample: $\theta = g(\lambda, \epsilon)$.
 - 2.3 Compute gradient estimate: $\hat{g} = \nabla_\lambda [\log f(\theta, x) - \log q_\lambda(\theta)]$.
 - 2.4 Update parameters: $\lambda \leftarrow \lambda + \rho \cdot \hat{g}$ (where ρ is learning rate).

- *Key Advantage:* We can compute $\hat{g}$ efficiently using automatic differentiation libraries (e.g., PyTorch, TensorFlow).

The "Zero-Forcing" Pathology

- While VI is computationally efficient, it has a known pathology: it systematically *underestimates posterior variance*.

The "Zero-Forcing" Pathology

- While VI is computationally efficient, it has a known pathology: it systematically *underestimates posterior variance*.
- **The Root Cause:** Recall that maximizing the ELBO minimizes the *reverse* KL divergence:

$$D_{KL}(q||p) = \mathbb{E}_q \left[\log \frac{q(\Theta)}{p(\Theta|x)} \right]$$

The "Zero-Forcing" Pathology

- While VI is computationally efficient, it has a known pathology: it systematically *underestimates posterior variance*.
- **The Root Cause:** Recall that maximizing the ELBO minimizes the *reverse* KL divergence:

$$D_{KL}(q||p) = \mathbb{E}_q \left[\log \frac{q(\Theta)}{p(\Theta|x)} \right]$$

- **The Infinite Penalty:** If $p(\theta|x)$ is near zero, but $q(\theta) > 0$, the ratio explodes. To avoid an infinite penalty, the algorithm is forced to set $q(\theta) \approx 0$ wherever $p(\theta|x)$ is low. This is called *zero-forcing*.

The "Zero-Forcing" Pathology

- While VI is computationally efficient, it has a known pathology: it systematically *underestimates posterior variance*.
- **The Root Cause:** Recall that maximizing the ELBO minimizes the *reverse* KL divergence:

$$D_{KL}(q||p) = \mathbb{E}_q \left[\log \frac{q(\Theta)}{p(\Theta|x)} \right]$$

- **The Infinite Penalty:** If $p(\theta|x)$ is near zero, but $q(\theta) > 0$, the ratio explodes. To avoid an infinite penalty, the algorithm is forced to set $q(\theta) \approx 0$ wherever $p(\theta|x)$ is low. This is called *zero-forcing*.
- **The Consequence:** If the true posterior is multi-modal, a simpler q will "choose" a single mode and fit tightly inside it, rather than spanning across low-probability valleys.
- *Takeaway:* VI gives you a fast, localized approximation, but it is often *overconfident* about its predictions.

Summary: Variational Inference

- **The Core Problem:** For most interesting models, the posterior $f(\theta|x)$ is *intractable* because computing the evidence $m(x)$ requires high-dimensional integration.
- **The VI Paradigm:** We transform *inference* into *optimization*. We find the closest approximation q^* within a family $\mathcal{Q}$ by maximizing the *ELBO* (Evidence Lower Bound).
- **Methods:**
 - **CAVI:** Provides *closed-form* updates for conjugate models (e.g., K-means).
 - **Black Box VI:** Uses *gradients* to optimize general families (e.g., Neural Networks).
- **The Reparameterization Trick:** Allows direct gradients through stochastic nodes, crucial for low-variance gradients in deep generative models.
- **Trade-off:** VI is *faster and more scalable* than MCMC, but it is an *approximation* and does not guarantee exactness.

The "Killer Apps" of Variational Inference

- Where do we actually use these algorithms in practice?

The "Killer Apps" of Variational Inference

- Where do we actually use these algorithms in practice?
- **Topic Modeling (e.g., LDA):**
 - Uses *CAVI* to discover hidden thematic structures in massive text corpora (e.g., millions of news articles) without human labels.

The "Killer Apps" of Variational Inference

- Where do we actually use these algorithms in practice?
- **Topic Modeling (e.g., LDA):**
 - Uses *CAVI* to discover hidden thematic structures in massive text corpora (e.g., millions of news articles) without human labels.
- **Variational Autoencoders (VAEs):**
 - Uses the *Reparameterization Trick* to train deep generative models. Used for synthesizing images, audio, and even discovering new molecular structures.

The "Killer Apps" of Variational Inference

- Where do we actually use these algorithms in practice?
- **Topic Modeling (e.g., LDA):**
 - Uses *CAVI* to discover hidden thematic structures in massive text corpora (e.g., millions of news articles) without human labels.
- **Variational Autoencoders (VAEs):**
 - Uses the *Reparameterization Trick* to train deep generative models. Used for synthesizing images, audio, and even discovering new molecular structures.
- **Bayesian Neural Networks (BNNs):**
 - Uses *Black Box VI* to place distributions over network weights. Provides crucial *uncertainty quantification* for safety-critical systems like autonomous driving and medical AI.